

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

OLEH:

ZAHRA NABILA

NIM 2310817320007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Zahra Nabila
NIM : 2310817320007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program	45
C. Pembahasan	46
D. Tautan Git.....	62

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	45
Gambar 2. Screenshot Hasil Jawaban Soal 1	45
Gambar 3. Screenshot Hasil Jawaban Soal 1	46
Gambar 4. Screenshot Hasil Jawaban Soal 1	46

DAFTAR TABEL

Table 1. Source Code Jawaban Soal 1	7
Table 2. Source Code Jawaban Soal 1	13
Table 3. Source Code Jawaban Soal 1	14
Table 4. Source Code Jawaban Soal 1	14
Table 5. Source Code Jawaban Soal 1	16
Table 6. Source Code Jawaban Soal 1	17
Table 7. Source Code Jawaban Soal 1	19
Table 8. Source Code Jawaban Soal 1	19
Table 9. Source Code Jawaban Soal 1	20
Table 10. Source Code Jawaban Soal 1	21
Table 11. Source Code Jawaban Soal 1	22
Table 12. Source Code Jawaban Soal 1	24
Table 13. Source Code Jawaban Soal 1	25
Table 14. Source Code Jawaban Soal 1	25
Table 15. Source Code Jawaban Soal 1	28
Table 16. Source Code Jawaban Soal 1	30
Table 17. Source Code Jawaban Soal 1	31
Table 18. Source Code Jawaban Soal 1	32
Table 19. Source Code Jawaban Soal 1	33
Table 20. Source Code Jawaban Soal 1	36
Table 21. Source Code Jawaban Soal 1	38
Table 22. Source Code Jawaban Soal 1	41
Table 23. Source Code Jawaban Soal 1	44

SOAL 1

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
 - b. Gunakan KotlinX Serialization sebagai library JSON.
 - c. Gunakan library seperti Coil atau Glide untuk image loading.
 - d. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDb) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
 - e. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
 - f. Gunakan caching strategy pada Room.
 - g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya:

A. Source Code

File AndroidManifest.xml	
1	<pre><?xml version="1.0" encoding="utf-8"?> <manifest xmlns:android="http://schemas.android.com/apk/res/android" xmlns:tools="http://schemas.android.com/tools"> <uses-permission android:name="android.permission.INTERNET" /></pre>
2	
3	
4	
5	
6	

7	<application
8	android:allowBackup="true"
9	android:dataExtractionRules="@xml/data_extraction_rules"
10	android:fullBackupContent="@xml/backup_rules"
11	android:icon="@mipmap/ic_launcher"
12	android:label="@string/app_name"
13	android:roundIcon="@mipmap/ic_launcher_round"
14	android:supportsRtl="true"
15	android:theme="@style/Theme.MovieList"
16	tools:targetApi="31">
17	<activity
18	android:name=".presentation.ui.activity.MainActivity"
19	android:exported="true">
20	<intent-filter>
21	<action
22	android:name="android.intent.action.MAIN" />
23	<category
24	android:name="android.intent.category.LAUNCHER" />
25	</intent-filter>
26	</activity>
27	<activity
28	android:name=".presentation.ui.activity.DetailActivity"
29	android:exported="false">
30	</activity>
31	</application>
32	
33	</manifest>

Table 1. Source Code Jawaban Soal 1

File MainActivity.kt	
1	package com.example.movielist.presentation.ui.activity
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import android.view.View
6	import android.widget.Toast
7	import androidx.activity.viewModels
8	import androidx.appcompat.app.AppCompatActivity
9	import androidx.appcompat.app.AppCompatActivityDelegate
10	import androidx.lifecycle.Observer
11	import androidx.recyclerview.widget.LinearLayoutManager
12	import androidx.room.Room
13	import
	com.example.movielist.data.local.MovieAppPreferences
14	import
	com.example.movielist.data.local.database.AppDatabase
15	import
	com.example.movielist.data.remote.api.RetrofitClient
16	import
	com.example.movielist.data.repository.MovieRepositoryIm pl
17	import
	com.example.movielist.databinding.ActivityMainBinding
18	import
	com.example.movielist.domain.usecase.GetPopularMoviesUs eCase
19	import
	com.example.movielist.presentation.ui.adapter.MovieAdap ter
20	import

	com.example.movielist.presentation.viewmodel.MovieViewM
	odel
21	import
	com.example.movielist.presentation.viewmodel.ViewModelF
	actory
22	import com.example.movielist.utils.Result
23	
24	class MainActivity : AppCompatActivity() {
25	
26	private lateinit var binding: ActivityMainBinding
27	private lateinit var movieAdapter: MovieAdapter
28	private lateinit var movieAppPreferences:
	MovieAppPreferences
29	
30	private val movieViewModel: MovieViewModel by
	<i>viewModels</i> {
31	val apiService = RetrofitClient.tmdbApiService
32	val database = Room.databaseBuilder(
33	<i>applicationContext</i> ,
34	AppDatabase::class.java,
35	AppDatabase.DATABASE_NAME
36	).build()
37	val movieDao = database.movieDao()
38	
39	val tmdbApiKey =
	"71819bfeac768c2a5b9a32b26e50cael"
40	movieAppPreferences.saveApiKey(tmdbApiKey)
41	
42	val movieRepositoryImpl =
	MovieRepositoryImpl(apiService, movieDao, tmdbApiKey)
43	val getPopularMoviesUseCase =

	GetPopularMoviesUseCase(movieRepositoryImpl)
44	ViewModelFactory(getPopularMoviesUseCase)
45	}
46	
47	override fun onCreate(savedInstanceState: Bundle?)
	{
48	super.onCreate(savedInstanceState)
49	binding =
	ActivityMainBinding.inflate(layoutInflater)
50	setContentView(binding.root)
51	
52	movieAppPreferences = MovieAppPreferences(this)
53	
54	setupRecyclerView()
55	observeViewModel()
56	setupDarkModeToggle()
57	
58	binding.btnRetry.setOnClickListener {
59	movieViewModel.fetchPopularMovies()
60	}
61	}
62	
63	private fun setupRecyclerView() {
64	movieAdapter = MovieAdapter()
65	binding.rvMovies.apply {
66	layoutManager =
	LinearLayoutManager(this@MainActivity)
67	adapter = movieAdapter
68	}
69	
70	movieAdapter.onItemClick = { movie ->

71	val intent = Intent(this,
	DetailActivity::class.java).apply {
72	putExtra(DetailActivity.EXTRA_MOVIE,
	movie)
73	}
74	startActivity(intent)
75	}
76	}
77	
78	private fun observeViewModel() {
79	movieViewModel.popularMovies.observe(this,
	Observer { result ->
80	when (result) {
81	is Result.Loading -> {
82	binding.progressBar.visibility =
	View.VISIBLE
83	binding.tvError.visibility =
	View.GONE
84	binding.btnRetry.visibility =
	View.GONE
85	binding.rvMovies.visibility =
	View.GONE
86	}
87	is Result.Success -> {
88	binding.progressBar.visibility =
	View.GONE
89	binding.tvError.visibility =
	View.GONE
90	binding.btnRetry.visibility =
	View.GONE
91	binding.rvMovies.visibility =

	<code>View.VISIBLE</code>
92	<code>movieAdapter.submitList(result.data)</code>
93	<code>}</code>
94	<code>is Result.Error -> {</code>
95	<code>binding.progressBar.visibility =</code>
	<code>View.GONE</code>
96	<code>binding.rvMovies.visibility =</code>
	<code>View.GONE</code>
97	<code>binding.tvError.visibility =</code>
	<code>View.VISIBLE</code>
98	<code>binding.btnRetry.visibility =</code>
	<code>View.VISIBLE</code>
99	<code>binding.tvError.text = "Error:</code> <code>\${result.exception.message}"</code>
100	<code>Toast.makeText(this, "Error:</code> <code>\${result.exception.message}", Toast.LENGTH_LONG).show()</code>
101	<code>}</code>
102	<code>}</code>
103	<code>})</code>
104	<code>}</code>
105	
106	<code>private fun setupDarkModeToggle() {</code>
107	<code>binding.switchDarkMode.isChecked =</code> <code>movieAppPreferences.getDarkModeState()</code>
108	
109	<code>applyTheme(movieAppPreferences.getDarkModeState())</code>
110	
111	<code>binding.switchDarkMode.setOnCheckedChangeListener</code> <code>{ _, isChecked -></code>
112	<code>movieAppPreferences.saveDarkModeState(isChecked)</code>
113	<code>applyTheme(isChecked)</code>

114	}
115	}
116	
117	private fun applyTheme(isDarkMode: Boolean) {
118	if (isDarkMode) {
119	AppCompatActivity.setDefaultNightMode (AppCompatActivity
	.MODE_NIGHT_YES)
120	} else {
121	AppCompatActivity.setDefaultNightMode (AppCompatActivity
	.MODE_NIGHT_NO)
122	}
123	}
124	}

Table 2. Source Code Jawaban Soal 1

File MovieDao.kt	
1	package com.example.movielist.data.local.dao
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import
	com.example.movielist.data.local.entities.MovieEntity
8	
9	@Dao
10	interface MovieDao {
11	@Insert(onConflict = OnConflictStrategy.REPLACE)
12	suspend fun insertAllMovies(movies: List<MovieEntity>)
13	
14	@Query("SELECT * FROM movies ORDER BY popularity DESC")

15	suspend fun getAllMovies(): List<MovieEntity>
16	
17	@Query("DELETE FROM movies")
18	suspend fun clearAllMovies()
19	}

Table 3. Source Code Jawaban Soal 1

File AppDatabase.kt	
1	Package com.example.movielist.data.local.database
2	
3	import androidx.room.Database
4	import androidx.room.RoomDatabase
5	import com.example.movielist.data.local.dao.MovieDao
6	import
	com.example.movielist.data.local.entities.MovieEntity
7	
8	@Database(entities = [MovieEntity::class], version = 1,
	exportSchema = false)
9	abstract class AppDatabase : RoomDatabase() {
10	abstract fun movieDao(): MovieDao
11	
12	companion object {
13	const val DATABASE_NAME = "tmdb_app_db"
14	}
15	}

Table 4. Source Code Jawaban Soal 1

File MovieEntity.kt	
1	package com.example.movielist.data.local.entities
2	
3	import androidx.room.Entity

```

4 import androidx.room.PrimaryKey
5 import com.example.movielist.domain.model.Movie
6
7 @Entity(tableName = "movies")
8 data class MovieEntity(
9     @PrimaryKey
10     val id: Int,
11     val title: String,
12     val overview: String,
13     val posterPath: String?,
14     val releaseDate: String,
15     val voteAverage: Double,
16     val popularity: Double
17 ) {
18     fun toDomainMovie(): Movie {
19         return Movie(
20             id = id,
21             title = title,
22             overview = overview,
23             posterPath = posterPath,
24             releaseDate = releaseDate,
25             voteAverage = voteAverage
26         )
27     }
28
29     companion object {
30         fun fromDomainMovie(movie: Movie, popularity:
31 Double): MovieEntity {
32             return MovieEntity(
33                 id = movie.id,
34                 title = movie.title,

```

34	overview = movie.overview,
35	posterPath = movie.posterPath,
36	releaseDate = movie.releaseDate,
37	voteAverage = movie.voteAverage,
38	popularity = popularity
39	)
40	}
41	}
42	}

Table 5. Source Code Jawaban Soal 1

File MovieAppPreferences.kt	
1	package com.example.movielist.data.local
2	
3	import android.content.Context
4	import android.content.SharedPreferences
5	
6	class MovieAppPreferences(context: Context) {
7	
8	private val sharedPreferences: SharedPreferences =
9	context.getSharedPreferences("tmdb_app_prefs",
10	Context.MODE_PRIVATE)
11	companion object {
12	private const val KEY_API_KEY = "api_key"
13	private const val KEY_DARK_MODE = "dark_mode"
14	}
15	
16	fun saveApiKey(apiKey: String) {
17	sharedPreferences.edit().putString(KEY_API_KEY,
18	apiKey).apply()
	}

19	
20	fun getApiKey(): String? {
21	return sharedPreferences.getString(KEY_API_KEY,
	null)
22	}
23	
24	fun saveDarkModeState(isDarkMode: Boolean) {
25	sharedPreferences.edit().putBoolean(KEY_DARK_MODE,
	isDarkMode).apply()
26	}
27	
28	fun getDarkModeState(): Boolean {
29	return
	sharedPreferences.getBoolean(KEY_DARK_MODE, false)
30	}
31	}

Table 6. Source Code Jawaban Soal 1

File RetrofitClient.kt	
1	package com.example.movielist.data.remote.api
2	
3	import
	com.jakewharton.retrofit2.converter.kotlinx.serialization
	.asConverterFactory
4	import kotlinx.serialization.json.Json
5	import okhttp3.MediaType.Companion.toMediaType
6	import okhttp3.OkHttpClient
7	import okhttp3.logging.HttpLoggingInterceptor
8	import retrofit2.Retrofit
9	import java.util.concurrent.TimeUnit
10	

```

11 object RetrofitClient {
12
13     private const val BASE_URL =
14         "https://api.themoviedb.org/3/"
15
16     private val json = Json {
17         ignoreUnknownKeys = true
18         prettyPrint = true
19     }
20
21     private val okHttpClient: OkHttpClient by lazy {
22         val logging = HttpLoggingInterceptor()
23         logging.setLevel(HttpLoggingInterceptor.Level.BODY)
24
25         OkHttpClient.Builder()
26             .addInterceptor(logging)
27             .connectTimeout(30, TimeUnit.SECONDS)
28             .readTimeout(30, TimeUnit.SECONDS)
29             .writeTimeout(30, TimeUnit.SECONDS)
30             .build()
31     }
32
33     val tmdbApiService: TmdbApiService by lazy {
34         Retrofit.Builder()
35             .baseUrl(BASE_URL)
36             .client(okHttpClient)
37             .addConverterFactory(json.asConverterFactory("application
38                 /json".toMediaType()))
39             .build()
40             .create(TmdbApiService::class.java)
41     }
42 }

```

40	<pre> } } </pre>
----	------------------------------

Table 7. Source Code Jawaban Soal 1

File TmdbApiService.xml	
1	package com.example.movielist.data.remote.api
2	
3	import com.example.movielist.data.remote.models.MovieListResponse
4	import retrofit2.Response
5	import retrofit2.http.GET
6	import retrofit2.http.Query
7	
8	interface TmdbApiService {
9	
10	@GET("movie/popular")
11	suspend fun getPopularMovies(12 @Query("api_key") apiKey: String, 13 @Query("language") language: String = "en-US", 14 @Query("page") page: Int = 1 15): Response<MovieListResponse>
16	}

Table 8. Source Code Jawaban Soal 1

File MovieDto.kt	
1	package com.example.movielist.data.remote.models
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	

6	@Serializable
7	data class MovieDto(
8	val adult: Boolean,
9	@SerializedName("backdrop_path")
10	val backdropPath: String?,
11	@SerializedName("genre_ids")
12	val genreIds: List<Int>,
13	val id: Int,
14	@SerializedName("original_language")
15	val originalLanguage: String,
16	@SerializedName("original_title")
17	val originalTitle: String,
18	val overview: String,
19	val popularity: Double,
20	@SerializedName("poster_path")
21	val posterPath: String?,
22	@SerializedName("release_date")
23	val releaseDate: String,
24	val title: String,
25	val video: Boolean,
26	@SerializedName("vote_average")
27	val voteAverage: Double,
28	@SerializedName("vote_count")
29	val voteCount: Int
30	)

Table 9. Source Code Jawaban Soal 1

File MovieDtoExtension.kt	
1	package com.example.movielist.data.remote.models
2	
3	import com.example.movielist.domain.model.Movie

4	import
	com.example.movielist.data.local.entities.MovieEntity
5	
6	fun MovieDto.toDomainMovie(): Movie {
7	return Movie(
8	id = id,
9	title = title,
10	overview = overview,
11	posterPath = posterPath,
12	releaseDate = releaseDate,
13	voteAverage = voteAverage
14	)
15	}
16	
17	fun MovieDto.toMovieEntity(): MovieEntity {
18	return MovieEntity(
19	id = id,
20	title = title,
21	overview = overview,
22	posterPath = posterPath,
23	releaseDate = releaseDate,
24	voteAverage = voteAverage,
25	popularity = popularity
26	)
27	}

Table 10. Source Code Jawaban Soal 1

File MovieListResponse.kt	
1	package com.example.movielist.data.remote.models
2	
3	import kotlinx.serialization.SerialName

4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class MovieListResponse(
8	val page: Int,
9	val results: List<MovieDto>,
10	@SerializedName("total_pages")
11	val totalPages: Int,
12	@SerializedName("total_results")
13	val totalResults: Int
14	)

Table 11. Source Code Jawaban Soal 1

File MovieRepository.kt	
1	package com.example.movielist.data.repository
2	
3	import com.example.movielist.data.local.dao.MovieDao
4	import
	com.example.movielist.data.remote.api.TmdbApiService
5	import
	com.example.movielist.data.remote.models.toDomainMovie
6	import
	com.example.movielist.data.remote.models.toMovieEntity
7	import com.example.movielist.domain.model.Movie
8	import com.example.movielist.utils.Result
9	import kotlinx.coroutines.flow.Flow
10	import kotlinx.coroutines.flow.flow
11	import retrofit2.HttpException
12	import java.io.IOException
13	
14	interface MovieRepository {

```

15     fun getPopularMovies(): Flow<Result<List<Movie>>>
16 }
17
18 class MovieRepositoryImpl(
19     private val apiService: TmdbApiService,
20     private val movieDao: MovieDao,
21     private val apiKey: String
22 ) : MovieRepository {
23
24     override fun getPopularMovies():
25     Flow<Result<List<Movie>>> = flow {
26
27         emit(Result.Loading)
28
29         val cachedMovies = movieDao.getAllMovies().map {
30             it.toDomainMovie() }
31
32         if (cachedMovies.isNotEmpty()) {
33             emit(Result.Success(cachedMovies))
34         }
35
36         try {
37             val response =
38             apiService.getPopularMovies(apiKey = apiKey)
39             if (response.isSuccessful) {
40                 val movieDtos = response.body()?.results
41                 ?: emptyList()
42                 val domainMovies = movieDtos.map {
43                     it.toDomainMovie() }
44
45                 movieDao.clearAllMovies()
46                 movieDao.insertAllMovies(movieDtos.map {
47                     it.toMovieEntity() })

```

40	
41	emit(Result.Success(domainMovies))
42	} else {
43	emit(Result.Error(Exception("API Error:"))
	\${response.code()} \${response.message()}"))
44	}
45	} catch (e: HttpException) {
46	emit(Result.Error(Exception("Network Error
	(HTTP \${e.code()}): \${e.message()}"))
47	} catch (e: IOException) {
48	emit(Result.Error(Exception("No Internet
	Connection or API Timeout: \${e.message()}"))
49	} catch (e: Exception) {
50	emit(Result.Error(Exception("An unexpected
	error occurred: \${e.localizedMessage()}"))
51	}
52	}
53	}

Table 12. Source Code Jawaban Soal 1

File Movie.kt	
1	package com.example.movielist.domain.model
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Movie(
8	val id: Int,
9	val title: String,
10	val overview: String,

11	val posterPath: String?,
12	val releaseDate: String,
13	val voteAverage: Double
14	) : Parcelable

Table 13. Source Code Jawaban Soal 1

File GetPopularMoviesUseCase.kt	
1	package com.example.movielist.domain.usecase
2	
3	import com.example.movielist.domain.model.Movie
4	import
	com.example.movielist.data.repository.MovieRepositoryImpl
5	import com.example.movielist.utils.Result
6	import kotlinx.coroutines.flow.Flow
7	
8	class GetPopularMoviesUseCase (
9	private val movieRepository: MovieRepositoryImpl
10	) {
11	operator fun invoke(): Flow<Result<List<Movie>>> {
12	return movieRepository.getPopularMovies()
13	}
14	}

Table 14. Source Code Jawaban Soal 1

File DetailActivity.kt	
1	package com.example.movielist.presentation.ui.activity
2	
3	import android.os.Build
4	import android.os.Bundle
5	import android.view.MenuItem
6	import android.widget.Toast

```

7  import androidx.appcompat.app.AppCompatActivity
8  import com.bumptech.glide.Glide
9  import
    com.example.movielist.databinding.ActivityDetailBinding
10 import com.example.movielist.domain.model.Movie
11
12
13 class DetailActivity : AppCompatActivity() {
14
15     private lateinit var binding: ActivityDetailBinding
16
17     companion object {
18         const val EXTRA_MOVIE = "extra_movie"
19     }
20
21     override fun onCreate(savedInstanceState: Bundle?) {
22         super.onCreate(savedInstanceState)
23         binding =
ActivityDetailBinding.inflate(layoutInflater)
24         setContentView(binding.root)
25
26         supportActionBar?.setDisplayHomeAsUpEnabled(true)
27
28         val movie = if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
29             intent.getParcelableExtra(EXTRA_MOVIE,
Movie::class.java)
30         } else {
31             @Suppress("DEPRECATION")
32             intent.getParcelableExtra(EXTRA_MOVIE)
33         }

```

```

34
35         movie?.let {
36
37             supportActionBar?.title = it.title
38
39             binding.apply {
40                 tvDetailTitle.text = it.title
41                 tvDetailReleaseDate.text = "Release Date:
42                 ${it.releaseDate}"
43                 tvDetailVoteAverage.text = "Rating:
44                 ${String.format("%.1f", it.voteAverage)}"
45                 tvDetailOverview.text = it.overview
46
47                 val imageUrl =
48                 "https://image.tmdb.org/t/p/w500${it.posterPath}"
49                 Glide.with(this@DetailActivity)
50                     .load(imageUrl)
51                     .centerCrop()
52                     .into(ivDetailPoster)
53             }
54         } ?: run {
55             Toast.makeText(this, "Film tidak ditemukan.",
56             Toast.LENGTH_SHORT).show()
57             finish()
58         }
59
60         override fun onOptionsItemSelected(item: MenuItem):
61         Boolean {
62             if (item.itemId == android.R.id.home) {
63                 onBackPressedDispatcher.onBackPressed()

```

60	return true
61	}
62	return super.onOptionsItemSelected(item)
63	}
64	}

Table 15. Source Code Jawaban Soal 1

File MovieAdapter.kt	
1	package com.example.movielist.presentation.ui.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.DiffUtil
6	import androidx.recyclerview.widget.ListAdapter
7	import androidx.recyclerview.widget.RecyclerView
8	import com.bumptech.glide.Glide
9	import com.example.movielist.databinding.ItemMovieBinding
10	import com.example.movielist.domain.model.Movie
11	
12	class MovieAdapter : ListAdapter<Movie,
	MovieAdapter.MovieViewHolder>(MovieDiffCallback()) {
13	
14	var onItemClick: ((Movie) -> Unit)? = null
15	
16	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): MovieViewHolder {
17	val binding =
	ItemMovieBinding.inflate(LayoutInflater.from(parent.conte
	xt), parent, false)
18	return MovieViewHolder(binding)
19	}

```

20
21     override fun onBindViewHolder(holder:
MovieViewHolder, position: Int) {
22         val movie = getItem(position)
23         holder.bind(movie)
24     }
25
26     inner class MovieViewHolder(private val binding:
ItemMovieBinding) :
27         RecyclerView.ViewHolder(binding.root) {
28
29         init {
30             binding.btnDetail.setOnClickListener {
31                 onItemClick?.invoke(getItem(adapterPosition))
32             }
33         }
34
35         fun bind(movie: Movie) {
36             binding.apply {
37                 tvMovieTitle.text = movie.title
38                 tvReleaseDate.text = "Release Date:
${movie.releaseDate}"
39                 tvVoteAverage.text = "Rating:
${String.format("%.1f", movie.voteAverage)}"
40                 tvOverview.text = movie.overview
41
42                 val imageUrl =
"https://image.tmdb.org/t/p/w500${movie.posterPath}"
43
44                 Glide.with(itemView.context)
45                     .load(imageUrl)

```

46	<code>.centerCrop()</code>
47	<code>.into(ivPoster)</code>
48	<code>}</code>
49	<code>}</code>
50	<code>}</code>
51	
52	<code>class MovieDiffCallback :</code>
	<code>DiffUtil.ItemCallback<Movie>() {</code>
53	<code> override fun areItemsTheSame(oldItem: Movie,</code>
	<code> newItem: Movie): Boolean {</code>
54	<code> return oldItem.id == newItem.id</code>
55	<code> }</code>
56	
57	<code> override fun areContentsTheSame(oldItem: Movie,</code>
	<code> newItem: Movie): Boolean {</code>
58	<code> return oldItem == newItem</code>
59	<code> }</code>
60	<code>}</code>
61	<code>}</code>

Table 16. Source Code Jawaban Soal 1

File MovieViewModel.kt	
1	<code>package com.example.movielist.presentation.viewmodel</code>
2	
3	<code>import androidx.lifecycle.LiveData</code>
4	<code>import androidx.lifecycle.MutableLiveData</code>
5	<code>import androidx.lifecycle.ViewModel</code>
6	<code>import androidx.lifecycle.viewModelScope</code>
7	<code>import com.example.movielist.domain.model.Movie</code>
8	<code>import</code>
	<code>com.example.movielist.domain.usecase.GetPopularMoviesUseC</code>

	ase
9	import com.example.movielist.utils.Result
10	import kotlinx.coroutines.launch
11	
12	class MovieViewModel (
13	private val getPopularMoviesUseCase:
	GetPopularMoviesUseCase
14	) : ViewModel() {
15	
16	private val _popularMovies =
	MutableLiveData<Result<List<Movie>>>()
17	val popularMovies: LiveData<Result<List<Movie>>> =
	_popularMovies
18	
19	init {
20	fetchPopularMovies()
21	}
22	
23	fun fetchPopularMovies() {
24	viewModelScope.launch {
25	getPopularMoviesUseCase().collect { result ->
26	_popularMovies.value = result
27	}
28	}
29	}
30	}

Table 17. Source Code Jawaban Soal 1

File ViewModelFactory.kt	
1	package com.example.movielist.presentation.viewmodel
2	

3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
	com.example.movielist.domain.usecase.GetPopularMoviesUseC
	ase
6	
7	class ViewModelFactory(
8	private val getPopularMoviesUseCase:
	GetPopularMoviesUseCase
9	) : ViewModelProvider.Factory {
10	
11	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
12	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java))
	{
13	@Suppress("UNCHECKED_CAST")
14	return
	MovieViewModel(getPopularMoviesUseCase) as T
15	}
16	throw IllegalArgumentException("Unknown ViewModel
	class")
17	}
18	}

Table 18. Source Code Jawaban Soal 1

File Result.kt	
1	package com.example.movielist.utils
2	
3	sealed class Result<out T> {
4	object Loading : Result<Nothing>()

5	data class Success<out T>(val data: T) : Result<T>()
6	data class Error(val exception: Exception) :
	Result<Nothing>()
7	}

Table 19. Source Code Jawaban Soal 1

File Activity_detail.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android
	"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.activity.DetailActivity">
8	
9	<androidx.constraintlayout.widget.ConstraintLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:padding="16dp">
13	
14	<ImageView
15	android:id="@+id/iv_detail_poster"
16	android:layout_width="0dp"
17	android:layout_height="300dp"
18	android:scaleType="centerCrop"
19	app:layout_constraintEnd_toEndOf="parent"
20	app:layout_constraintStart_toStartOf="parent"
21	app:layout_constraintTop_toTopOf="parent"
22	tools:src="@tools:sample/avatars" />

```

23
24         <TextView
25             android:id="@+id/tv_detail_title"
26             android:layout_width="0dp"
27             android:layout_height="wrap_content"
28             android:layout_marginTop="16dp"
29             android:textSize="24sp"
30             android:textStyle="bold"
31             app:layout_constraintEnd_toEndOf="parent"
32             app:layout_constraintStart_toStartOf="parent"
33 app:layout_constraintTop_toBottomOf="@id/iv_detail_poster
34 "
35
36             tools:text="Movie Title on Detail Page" />
37
38         <TextView
39             android:id="@+id/tv_detail_release_date"
40             android:layout_width="0dp"
41             android:layout_height="wrap_content"
42             android:layout_marginTop="8dp"
43             android:textSize="16sp"
44             app:layout_constraintEnd_toEndOf="parent"
45             app:layout_constraintStart_toStartOf="parent"
46 app:layout_constraintTop_toBottomOf="@id/tv_detail_title"
47             tools:text="Release Date: 2023-01-01" />
48
49         <TextView
50             android:id="@+id/tv_detail_vote_average"
51             android:layout_width="0dp"
52             android:layout_height="wrap_content"
53             android:layout_marginTop="4dp"
54             android:textSize="16sp"

```

53	app:layout_constraintEnd_toEndOf="parent"
54	app:layout_constraintStart_toStartOf="parent"
55	app:layout_constraintTop_toBottomOf="@id/tv_detail_release_date"
56	tools:text="Rating: 7.5" />
57	
58	<TextView
59	android:id="@+id/tv_detail_overview_label"
60	android:layout_width="0dp"
61	android:layout_height="wrap_content"
62	android:layout_marginTop="16dp"
63	android:text="Overview:"
64	android:textSize="18sp"
65	android:textStyle="bold"
66	app:layout_constraintEnd_toEndOf="parent"
67	app:layout_constraintStart_toStartOf="parent"
68	app:layout_constraintTop_toBottomOf="@id/tv_detail_vote_average" />
69	
70	<TextView
71	android:id="@+id/tv_detail_overview"
72	android:layout_width="0dp"
73	android:layout_height="wrap_content"
74	android:layout_marginTop="8dp"
75	android:textSize="16sp"
76	app:layout_constraintEnd_toEndOf="parent"
77	app:layout_constraintStart_toStartOf="parent"
78	app:layout_constraintTop_toBottomOf="@id/tv_detail_overview_label"
79	tools:text="This is a very long and detailed overview of the movie. It covers the plot, characters,

	themes, and critical reception." />
80	
81	</androidx.constraintlayout.widget.ConstraintLayout>
82	</ScrollView>

Table 20. Source Code Jawaban Soal 1

File Activity_main.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android "
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.activity.MainActivity">
8	
9	<TextView
10	android:id="@+id/tv_title"
11	android:layout_width="wrap_content"
12	android:layout_height="wrap_content"
13	android:layout_marginTop="8dp"
14	android:text="Popular Movies"
15	android:textSize="24sp"
16	android:textStyle="bold"
17	app:layout_constraintHorizontal_bias="0.454"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintTop_toTopOf="parent" />
20	
21	<ProgressBar
22	android:id="@+id/progress_bar"

```

23         android:layout_width="wrap_content"
24         android:layout_height="wrap_content"
25         android:visibility="gone"
26     app:layout_constraintTop_toBottomOf="@id/tv_title"
27     app:layout_constraintStart_toStartOf="parent"
28     app:layout_constraintEnd_toEndOf="parent"
29     app:layout_constraintBottom_toBottomOf="parent"
30     />
31     <TextView
32         android:id="@+id/tv_error"
33         android:layout_width="wrap_content"
34         android:layout_height="wrap_content"
35         android:text="Error: Could not load movies."
36         android:textColor="@android:color/holo_red_dark"
37         android:visibility="gone"
38         android:gravity="center"
39     app:layout_constraintTop_toBottomOf="@id/tv_title"
40     app:layout_constraintStart_toStartOf="parent"
41     app:layout_constraintEnd_toEndOf="parent"
42     app:layout_constraintBottom_toBottomOf="parent"
43     />
44     <Button
45         android:id="@+id/btn_retry"
46         android:layout_width="wrap_content"
47         android:layout_height="wrap_content"
48         android:text="Retry"
49         android:visibility="gone"
50         android:layout_marginTop="8dp"
51     app:layout_constraintTop_toBottomOf="@id/tv_error"

```

52	app:layout_constraintStart_toStartOf="parent"
53	app:layout_constraintEnd_toEndOf="parent"
54	app:layout_constraintBottom_toBottomOf="parent"
	/>
55	
56	<androidx.recyclerview.widget.RecyclerView
57	android:id="@+id/rv_movies"
58	android:layout_width="0dp"
59	android:layout_height="0dp"
60	android:layout_marginTop="16dp"
61	app:layout_constraintBottom_toBottomOf="parent"
62	app:layout_constraintEnd_toEndOf="parent"
63	app:layout_constraintStart_toStartOf="parent"
64	app:layout_constraintTop_toBottomOf="@+id/tv_title"
65	tools:listitem="@layout/item_movie" />
66	
67	<com.google.android.material.switchmaterial.SwitchMaterial
	1
68	android:id="@+id/switch_dark_mode"
69	android:layout_width="wrap_content"
70	android:layout_height="wrap_content"
71	android:layout_marginEnd="8dp"
72	android:layout_marginBottom="8dp"
73	android:text="Dark Mode"
74	app:layout_constraintEnd_toEndOf="parent"
75	app:layout_constraintTop_toTopOf="parent" />
76	
77	</androidx.constraintlayout.widget.ConstraintLayout>

Table 21. Source Code Jawaban Soal 1

File item_movie.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
	xmlns:android="http://schemas.android.com/apk/res/android
	"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="8dp"
9	app:cardElevation="4dp">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="16dp">
15	
16	<ImageView
17	android:id="@+id/iv_poster"
18	android:layout_width="550dp"
19	android:layout_height="150dp"
20	android:scaleType="centerCrop"
21	app:layout_constraintStart_toStartOf="parent"
22	app:layout_constraintTop_toTopOf="parent"
23	tools:src="@tools:sample/avatars" />
24	
25	<TextView
26	android:id="@+id/tv_movie_title"
27	android:layout_width="0dp"
28	android:layout_height="wrap_content"

```

29         android:layout_marginStart="16dp"
30         android:textStyle="bold"
31         android:textSize="18sp"
32         app:layout_constraintEnd_toEndOf="parent"
33     app:layout_constraintStart_toEndOf="@id/iv_poster"
34     app:layout_constraintTop_toTopOf="@id/iv_poster"
35     tools:text="Movie Title" />
36
37     <TextView
38         android:id="@+id/tv_release_date"
39         android:layout_width="0dp"
40         android:layout_height="wrap_content"
41         android:layout_marginStart="16dp"
42         android:layout_marginTop="4dp"
43         android:textSize="14sp"
44         app:layout_constraintEnd_toEndOf="parent"
45         app:layout_constraintStart_toEndOf="@id/iv_poster"
46     app:layout_constraintTop_toBottomOf="@id/tv_movie_title"
47         tools:text="Release Date: 2023-01-01" />
48
49     <TextView
50         android:id="@+id/tv_vote_average"
51         android:layout_width="0dp"
52         android:layout_height="wrap_content"
53         android:layout_marginStart="16dp"
54         android:layout_marginTop="4dp"
55         android:textSize="14sp"
56         app:layout_constraintEnd_toEndOf="parent"
57         app:layout_constraintStart_toEndOf="@id/iv_poster"
58     app:layout_constraintTop_toBottomOf="@id/tv_release_date"
59         tools:text="Rating: 7.5" />

```


60	
61	<TextView
62	android:id="@+id/tv_overview"
63	android:layout_width="0dp"
64	android:layout_height="wrap_content"
65	android:layout_marginTop="8dp"
66	android:maxLines="3"
67	android:ellipsize="end"
68	app:layout_constraintEnd_toEndOf="parent"
69	app:layout_constraintStart_toStartOf="parent"
70	app:layout_constraintTop_toBottomOf="@id/iv_poster"
71	tools:text="This is a short overview of the
	movie. It talks about the plot and characters..." />
72	
73	<Button
74	android:id="@+id/btn_detail"
75	android:layout_width="wrap_content"
76	android:layout_height="wrap_content"
77	android:layout_marginTop="16dp"
78	android:text="Lihat Detail"
79	app:layout_constraintEnd_toEndOf="parent"
80	app:layout_constraintTop_toBottomOf="@id/tv_overview"
	/>
81	
82	</androidx.constraintlayout.widget.ConstraintLayout>
83	</androidx.cardview.widget.CardView>

Table 22. Source Code Jawaban Soal 1

File build.gradle.kts (Module:app)	
1	plugins {
2	alias(libs.plugins.android.application)

```

3      alias(libs.plugins.kotlin.android)
4      id("kotlin-parcelize")
5      id("kotlin-kapt")
6      kotlin("plugin.serialization") version "2.0.21"
7  }
8
9  android {
10     namespace = "com.example.movielist"
11     compileSdk = 35
12
13     defaultConfig {
14         applicationId = "com.example.movielist"
15         minSdk = 24
16         targetSdk = 35
17         versionCode = 1
18         versionName = "1.0"
19
20         testInstrumentationRunner =
21         "androidx.test.runner.AndroidJUnitRunner"
22     }
23     buildTypes {
24         release {
25             isMinifyEnabled = false
26             proguardFiles(
27                 getDefaultProguardFile("proguard-
28                 android-optimize.txt"),
29                 "proguard-rules.pro"
30             )
31         }
32     }

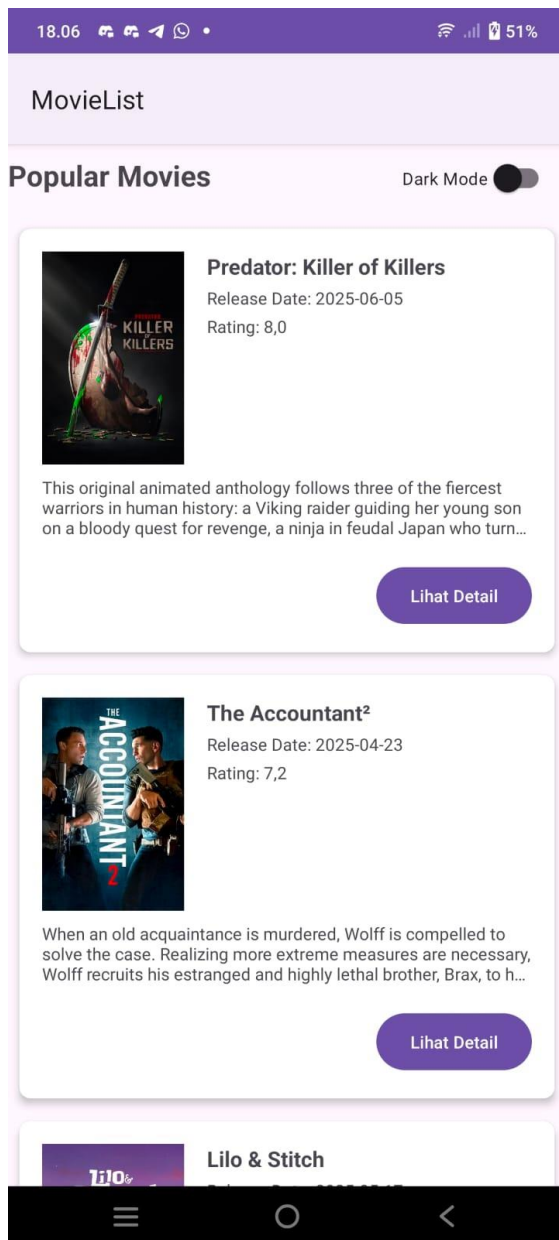
```

```
32     compileOptions {
33         sourceCompatibility = JavaVersion.VERSION_11
34         targetCompatibility = JavaVersion.VERSION_11
35     }
36     kotlinOptions {
37         jvmTarget = "11"
38     }
39     buildFeatures {
40         viewBinding = true
41     }
42 }
43
44 dependencies {
45
46     implementation(libs.androidx.core.ktx)
47     implementation(libs.androidx.appcompat)
48     implementation(libs.material)
49     implementation(libs.androidx.activity)
50     implementation(libs.androidx.constraintlayout)
51     testImplementation(libs.junit)
52     androidTestImplementation(libs.androidx.junit)
53     androidTestImplementation(libs.androidx.espresso.core)
54
55     // Room
56     implementation("androidx.room:room-runtime:2.6.1")
57     implementation("androidx.room:room-ktx:2.6.1")
58     kapt("androidx.room:room-compiler:2.6.1")
59
60     // Glide
61     implementation
62     ("com.github.bumptech.glide:glide:4.16.0")
```

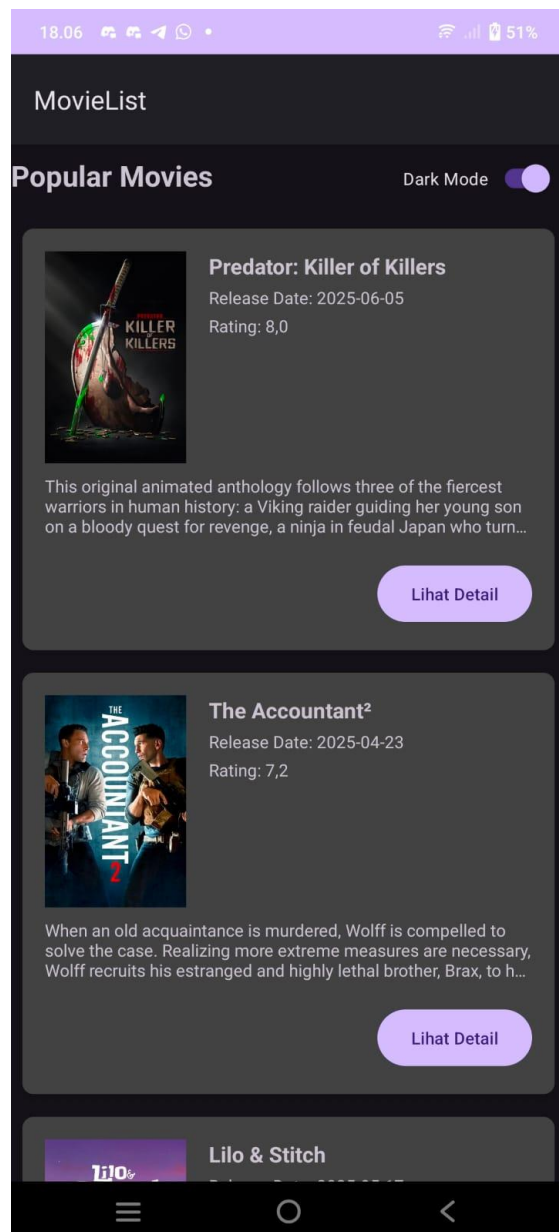
62	
63	// Retrofit & Kotlinx Serialization
64	implementation("com.squareup.retrofit2:retrofit:2.9.0"
	)
65	implementation("com.jakewharton.retrofit:retrofit2-
	kotlinx-serialization-converter:0.8.0")
66	implementation("org.jetbrains.kotlinx:kotlinx-
	serialization-json:1.6.3")
67	implementation("com.squareup.okhttp3:okhttp:4.9.3")
68	implementation("com.squareup.okhttp3:logging-
	interceptor:4.9.3")
69	implementation("com.jakewharton.retrofit:retrofit2-
	kotlinx-serialization-converter:1.0.0")
70	
71	// Lifecycle
72	implementation("androidx.lifecycle:lifecycle-
	viewmodel-ktx:2.7.0")
73	implementation("androidx.lifecycle:lifecycle-
	livedata-ktx:2.7.0")
74	
75	// Coroutine
76	implementation("org.jetbrains.kotlinx:kotlinx-
	coroutines-android:1.7.3")
77	
78	// Navigation
79	val nav_version = "2.8.9"
80	implementation("androidx.navigation:navigation-
	fragment:\$nav_version")
81	implementation("androidx.navigation:navigation-
	ui:\$nav_version")
82	}

Table 23. Source Code Jawaban Soal 1

B. Output Program



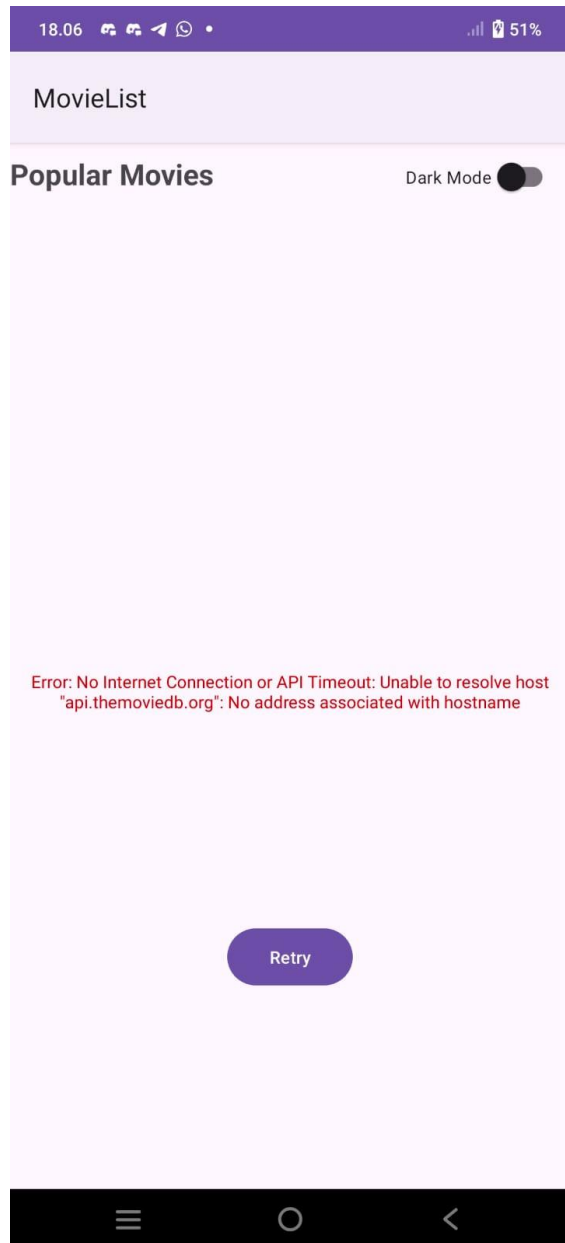
Gambar 1. Screenshot Hasil Jawaban Soal 1



Gambar 2. Screenshot Hasil Jawaban Soal 1



Gambar 3. Screenshot Hasil Jawaban Soal 1



Gambar 4. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

File AndroidManifest.xml

File **AndroidManifest.xml** di atas adalah komponen penting dalam aplikasi Android karena mendeklarasikan izin, aktivitas, dan konfigurasi penting aplikasi kepada sistem Android. Pada bagian atas, terdapat deklarasi XML standar dengan encoding UTF-8, serta dua namespace: **xmlns:android** untuk atribut Android standar, dan

xmlns:tools untuk konfigurasi yang digunakan oleh Android tools seperti Android Studio.

Bagian pertama dari isi manifest adalah deklarasi izin aplikasi dengan **<uses-permission android:name="android.permission.INTERNET" />**, yang menyatakan bahwa aplikasi memerlukan akses ke internet. Izin ini sangat penting, terutama jika aplikasi menggunakan API eksternal, seperti The Movie Database (TMDB), untuk mengambil data secara online.

Selanjutnya adalah elemen **<application>** yang menyimpan konfigurasi inti dari aplikasi, seperti ikon (**android:icon** dan **android:roundIcon**), label aplikasi (**android:label**), tema (**android:theme**), serta pengaturan pencadangan seperti **android:allowBackup**, **android:dataExtractionRules**, dan **android:fullBackupContent**, yang berkaitan dengan bagaimana data aplikasi dicadangkan dan dipulihkan di perangkat Android yang mendukungnya. Atribut **tools:targetApi="31"** menunjukkan bahwa alat pengembangan (tools) seperti lint akan menargetkan API level 31 untuk pemeriksaan dan validasi.

Di dalam elemen **<application>** terdapat dua **<activity>** yang dideklarasikan. Aktivitas utama (**MainActivity**) didefinisikan dengan **android:exported="true"** yang artinya aktivitas ini dapat diakses dari luar aplikasi, dan di dalamnya terdapat **<intent-filter>** dengan aksi **MAIN** dan kategori **LAUNCHER**, yang menjadikan **MainActivity** sebagai titik masuk utama aplikasi ketika ikon aplikasi diklik di layar utama perangkat. Aktivitas kedua adalah **DetailActivity**, yang dideklarasikan dengan **android:exported="false"**, artinya hanya bisa diakses secara internal dari dalam aplikasi, bukan dari aplikasi atau komponen eksternal lain. Ini sesuai dengan praktik keamanan Android yang membatasi akses aktivitas sensitif atau tidak diperlukan dari luar aplikasi.

File MainActivity.kt

Kelas **MainActivity** pada kode di atas merupakan *entry point* utama dalam aplikasi Android "MovieList" yang menggunakan arsitektur modern dengan *MVVM* (*Model-View-ViewModel*), *ViewBinding*, dan berbagai komponen arsitektur Android

seperti Room, Retrofit, dan LiveData. Kelas ini berperan sebagai lapisan *presentation* yang menghubungkan tampilan antarmuka pengguna dengan logika aplikasi.

Pada awal kelas, terdapat beberapa deklarasi penting. Objek **binding** digunakan untuk mengakses elemen UI melalui ViewBinding tanpa menggunakan **findViewById**, sehingga meningkatkan keamanan dan efisiensi. **movieAdapter** adalah adapter untuk **RecyclerView** yang digunakan menampilkan daftar film, dan **movieAppPreferences** adalah wrapper untuk **SharedPreferences** yang digunakan menyimpan preferensi pengguna seperti dark mode dan API key.

ViewModel **movieViewModel** dibuat menggunakan delegasi by **viewModels** dan menyertakan custom **ViewModelFactory**. Dalam blok inisialisasi ViewModel tersebut, dilakukan beberapa hal: menginisialisasi Retrofit untuk API TMDb, membuat instansi Room untuk database lokal, menyimpan API key ke dalam **SharedPreferences**, dan membuat repository untuk menyatukan data dari API dan database.

Di dalam **onCreate()**, **binding** diinisialisasi dan root view diset ke konten tampilan. **movieAppPreferences** kemudian diinisialisasi dengan konteks saat ini. Fungsi **setupRecyclerView()** dipanggil untuk mengatur tampilan daftar film menggunakan **LinearLayoutManager** dan **MovieAdapter**, serta menyediakan aksi klik pada item untuk membuka **DetailActivity** dengan intent dan data film terpilih. Fungsi **observeViewModel()** mengamati perubahan LiveData **popularMovies** dari ViewModel dan menyesuaikan tampilan UI sesuai status **Result** (*Loading, Success, Error*). Saat berhasil, data film ditampilkan di RecyclerView; jika gagal, pesan error ditampilkan bersama tombol retry.

Fitur dark mode diatur melalui **setupDarkModeToggle()**, yang membaca preferensi sebelumnya dan mengatur status **Switch** serta menerapkan tema sesuai keadaan. Listener **setOnCheckedChangeListener** disiapkan untuk menyimpan preferensi baru saat switch diklik dan menerapkan tema yang sesuai menggunakan **AppCompatDelegate**.

File MovieDao.kt

File **MovieDao.kt** merupakan komponen penting dalam arsitektur lokal aplikasi Android yang menggunakan Room Database. DAO (*Data Access Object*) ini berfungsi sebagai antarmuka untuk mengakses dan memanipulasi data film yang tersimpan secara lokal di tabel **movies**. Antarmuka ini ditandai dengan anotasi **@Dao**, yang memberi tahu Room bahwa ini adalah komponen yang menyediakan perintah SQL untuk entitas **MovieEntity**. Terdapat tiga fungsi utama di dalamnya. Pertama, fungsi **insertAllMovies** memungkinkan penyisipan daftar film ke dalam database dengan strategi konflik **REPLACE**, artinya data lama akan digantikan jika terjadi duplikasi berdasarkan primary key—fitur ini berguna untuk memastikan data film yang ditampilkan selalu yang terbaru saat diambil dari API. Kedua, fungsi **getAllMovies** digunakan untuk mengambil seluruh data film dari tabel, yang kemudian diurutkan berdasarkan tingkat popularitas secara menurun. Fungsi ini bersifat **suspend**, yang berarti dijalankan secara asynchronous menggunakan coroutine, sehingga tidak membebani thread utama. Ketiga, fungsi **clearAllMovies** menghapus seluruh data dari tabel **movies**, yang biasanya digunakan saat penyegaran data atau pembersihan lokal cache.

File AppDatabase.kt

File **AppDatabase.kt** adalah komponen utama yang mendefinisikan *Room Database* dalam aplikasi Android "MovieList". Kelas ini ditandai dengan anotasi **@Database**, yang memberi tahu sistem bahwa ini adalah kelas database dan mengatur konfigurasi penting seperti entitas yang digunakan (**MovieEntity**), versi database (**version = 1**), dan pengaturan **exportSchema = false** untuk mencegah ekspor skema database ke file JSON, yang biasanya dilakukan untuk tujuan dokumentasi atau pengujian. **MovieEntity** sendiri merupakan representasi tabel **movies** dalam database lokal.

Kelas **AppDatabase** diturunkan dari **RoomDatabase**, dan bersifat abstrak karena berfungsi sebagai kerangka kerja yang akan digunakan Room untuk menghasilkan implementasi konkret di balik layar. Di dalamnya terdapat satu fungsi abstrak **movieDao()**, yang digunakan untuk mengakses DAO (*Data Access Object*)

MovieDao. Fungsi ini menjadi pintu utama untuk menjalankan operasi database seperti menyisipkan, mengambil, dan menghapus data film.

Selain itu, terdapat **companion object** yang mendeklarasikan konstanta **DATABASE_NAME** dengan nilai **"tmdb_app_db"**, yang digunakan sebagai nama file database yang disimpan di perangkat pengguna. Penamaan ini penting untuk identifikasi dan pengelolaan database, terutama saat terjadi pembaruan versi atau migrasi skema.

File **MovieEntity.kt**

File **MovieEntity.kt** mendefinisikan *entitas* Room untuk menyimpan data film di database lokal aplikasi "MovieList". Kelas data **MovieEntity** ditandai dengan anotasi **@Entity**, yang menunjukkan bahwa kelas ini merepresentasikan sebuah tabel bernama **"movies"** di database Room. Setiap properti dalam kelas ini menjadi kolom pada tabel tersebut, di mana **id** berfungsi sebagai **@PrimaryKey**, artinya merupakan identifikasi unik bagi setiap entri film. Properti lainnya seperti **title**, **overview**, **posterPath**, **releaseDate**, **voteAverage**, dan **popularity** berisi informasi detail mengenai film yang disimpan.

Selain menyimpan data, **MovieEntity** juga menyediakan dua fungsi penting untuk konversi data antar lapisan: fungsi **toDomainMovie()** dan **fromDomainMovie(...)**. Fungsi **toDomainMovie()** digunakan untuk mengubah **MovieEntity** menjadi objek **Movie** dari lapisan domain, yaitu representasi logika bisnis yang digunakan ViewModel dan UI. Sebaliknya, fungsi **fromDomainMovie()** digunakan untuk membentuk **MovieEntity** dari objek **Movie** dengan tambahan nilai **popularity**, sehingga data dari domain dapat disimpan kembali ke database. Konversi dua arah ini mencerminkan prinsip *separation of concerns*, di mana data yang digunakan di database (*data layer*) dipisahkan dari data yang digunakan di antarmuka dan logika aplikasi (*domain layer*).

File **MovieAppPreferences.kt**

File **MovieAppPreferences.kt** berfungsi sebagai kelas utilitas untuk mengelola penyimpanan data sederhana secara lokal menggunakan

SharedPreferences pada aplikasi Android "MovieList". **SharedPreferences** sendiri adalah mekanisme bawaan Android yang memungkinkan penyimpanan data *key-value* secara permanen di perangkat, cocok untuk data ringan seperti preferensi pengguna atau konfigurasi aplikasi.

Dalam kelas ini, objek **SharedPreferences** diinisialisasi menggunakan nama **"tmdb_app_prefs"** dengan mode **Context.MODE_PRIVATE**, yang memastikan bahwa data hanya bisa diakses oleh aplikasi itu sendiri. Terdapat dua kunci utama yang dikelola dalam kelas ini, yaitu **KEY_API_KEY** untuk menyimpan API key TMDB, dan **KEY_DARK_MODE** untuk menyimpan status mode gelap (*dark mode*).

Fungsi **saveApiKey(apiKey: String)** dan **getApiKey()** digunakan untuk menyimpan dan mengambil nilai API key yang diperlukan untuk autentikasi saat melakukan permintaan ke server TMDB. Sementara itu, fungsi **saveDarkModeState(isDarkMode: Boolean)** dan **getDarkModeState()** digunakan untuk menyimpan dan membaca preferensi pengguna terhadap penggunaan mode gelap. Dengan menyimpan status ini, aplikasi dapat mempertahankan tema yang dipilih pengguna meskipun aplikasi ditutup atau perangkat di-restart.

File RetrofitClient.kt

File **RetrofitClient.kt** berperan sebagai *singleton object* yang mengatur dan menyediakan instance Retrofit untuk melakukan komunikasi jaringan dengan API TMDB (*The Movie Database*) dalam aplikasi "MovieList". Kelas ini mendefinisikan konfigurasi dasar dan penting untuk memastikan proses permintaan dan pengambilan data dari server berjalan dengan efisien, andal, dan mudah untuk di-*debug*.

Pertama, konstanta **BASE_URL** menyimpan alamat dasar endpoint API TMDB versi 3, yang akan menjadi dasar semua permintaan jaringan. Kemudian, **Json** dikonfigurasi dari *KotlinX Serialization* untuk parsing JSON, dengan opsi **ignoreUnknownKeys = true** agar proses deserialisasi tidak gagal jika ada kunci asing yang tidak dikenali dalam respons API. **prettyPrint = true** digunakan untuk memformat hasil JSON agar lebih mudah dibaca, meskipun ini lebih berdampak saat melakukan *debugging*.

Untuk lapisan HTTP, digunakan **OkHttpClient**, yang dikonfigurasi secara *lazy* untuk efisiensi. Salah satu fitur penting yang ditambahkan adalah **HttpLoggingInterceptor** yang mencetak log isi permintaan dan respons HTTP ke logcat, sangat berguna untuk keperluan debugging saat mengembangkan aplikasi. Selain itu, waktu tunggu koneksi (**connectTimeout**, **readTimeout**, dan **writeTimeout**) disetel ke 30 detik untuk menangani kondisi jaringan yang lambat.

Akhirnya, objek **tmdbApiService** dibuat secara *lazy* menggunakan **Retrofit.Builder**, yang menghubungkan semua konfigurasi: **baseUrl**, **okHttpClient**, dan **converterFactory** untuk JSON. Hasilnya adalah instance dari **TmdbApiService**, yaitu antarmuka yang berisi definisi endpoint API yang akan digunakan oleh aplikasi.

File TmdbApiService.kt

File **TmdbApiService.kt** mendefinisikan antarmuka Retrofit yang berisi deklarasi endpoint untuk melakukan permintaan data ke API *The Movie Database* (TMDB). Antarmuka ini menjadi kontrak antara aplikasi dan layanan web TMDB, yang mendeskripsikan bagaimana permintaan dibuat dan respons apa yang diharapkan.

Dalam antarmuka ini terdapat satu fungsi utama, **getPopularMovies()**, yang ditandai dengan anotasi **@GET("movie/popular")**. Anotasi ini menunjukkan bahwa fungsi akan melakukan permintaan HTTP GET ke endpoint **movie/popular**, yang merupakan bagian dari TMDB API untuk mengambil daftar film populer.

Fungsi **getPopularMovies()** bersifat **suspend**, artinya dapat dipanggil dalam korutin sehingga mendukung pemrograman *asynchronous* tanpa memblokir *main thread*, sangat penting untuk menjaga responsivitas UI. Fungsi ini menerima tiga parameter: **apiKey** untuk otentikasi permintaan ke TMDB, **language** untuk menentukan bahasa hasil (secara default diatur ke **"en-US"**), dan **page** untuk mendukung pagination data (default halaman 1). Ketiganya akan secara otomatis dikonversi menjadi parameter URL melalui anotasi **@Query**.

Hasil dari fungsi ini adalah objek **Response<MovieListResponse>**, di mana **MovieListResponse** merupakan model data yang merepresentasikan struktur respons

JSON dari TMDB untuk daftar film populer. Dengan membungkus respons dalam kelas **Response<>**, aplikasi dapat memeriksa status HTTP dan isi respons secara eksplisit, seperti sukses atau error, sebelum memproses datanya lebih lanjut.

File **MovieDto.kt**

File **MovieDto.kt** merupakan bagian dari lapisan data pada aplikasi Android yang berfungsi sebagai *Data Transfer Object* (DTO) untuk memetakan respons JSON dari API *The Movie Database* (TMDB) ke dalam objek Kotlin. Kelas ini diberi anotasi **@Serializable** dari pustaka Kotlinx Serialization, yang memungkinkan proses serialisasi dan deserialisasi data JSON secara otomatis ketika berkomunikasi dengan jaringan menggunakan Retrofit. Setiap properti di dalam **MovieDto** merepresentasikan elemen informasi dari sebuah film, seperti **id**, **title**, **overview**, **popularity**, dan **releaseDate**. Beberapa properti diberi anotasi **@SerializedName** untuk menyelaraskan nama-nama variabel Kotlin dengan kunci pada JSON, misalnya **posterPath** disesuaikan dengan **poster_path** dari API. Selain itu, atribut seperti **genreIds**, **originalLanguage**, dan **video** menambah informasi tambahan mengenai kategori, bahasa asli, dan jenis konten. Tujuan utama dari **MovieDto** adalah menjadi representasi awal data film yang diambil dari API, sebelum nantinya dikonversi ke dalam model domain internal (seperti **Movie**) guna memisahkan struktur eksternal dari logika bisnis aplikasi. Hal ini membantu menjaga arsitektur aplikasi tetap bersih, modular, dan mudah untuk dikembangkan atau diperbarui ketika struktur API berubah.

File **MovieDtoExtension.kt**

File **MovieDtoExtension.kt** berisi dua buah *extension function* pada objek **MovieDto** yang berfungsi untuk mengonversi data hasil dari respons API (yang berbentuk objek **MovieDto**) ke dua bentuk lain dalam arsitektur aplikasi, yaitu ke dalam model domain (**Movie**) dan ke dalam entitas lokal (**MovieEntity**) yang disimpan di database. Fungsi **toDomainMovie()** mengonversi **MovieDto** menjadi objek **Movie**, yaitu representasi data film yang digunakan di lapisan domain atau logika bisnis aplikasi. Proses ini melibatkan pemetaan properti penting seperti **id**, **title**, **overview**,

posterPath, **releaseDate**, dan **voteAverage**, tanpa membawa informasi yang hanya relevan untuk penyimpanan atau transport, seperti **popularity**. Sementara itu, fungsi **toMovieEntity()** mengonversi **MovieDto** menjadi **MovieEntity**, yaitu representasi data film yang akan disimpan secara lokal menggunakan Room. Fungsi ini mencakup semua atribut penting termasuk **popularity**, karena nilai ini digunakan untuk proses penyortiran di database lokal. Dengan adanya dua fungsi konversi ini, aplikasi dapat memisahkan dan menjaga integritas data antara lapisan data, domain, dan penyimpanan, sesuai dengan prinsip *Clean Architecture*, sekaligus memudahkan pemeliharaan dan pengembangan aplikasi di masa depan.

File **MovieListResponse.kt**

File **MovieListResponse.kt** merupakan representasi dari struktur respons JSON yang diterima dari endpoint API populer TMDB yang mengembalikan daftar film. Kelas ini diberi anotasi **@Serializable** dari pustaka **kotlinx.serialization**, yang memudahkan proses konversi dari data JSON ke objek Kotlin saat menerima respons jaringan melalui Retrofit. Di dalam kelas ini terdapat empat properti utama yang sesuai dengan struktur data dari respons TMDB: **page** menunjukkan nomor halaman saat ini, **results** berisi daftar objek **MovieDto** yang merupakan detail film individual, **totalPages** adalah jumlah total halaman yang tersedia untuk permintaan tersebut, dan **totalResults** mencerminkan jumlah keseluruhan data film yang tersedia. Anotasi **@SerializedName** digunakan untuk menyesuaikan nama properti Kotlin (**totalPages**, **totalResults**) dengan nama yang digunakan di dalam JSON (**total_pages**, **total_results**), karena notasi pada API biasanya menggunakan gaya *snake_case*. Kelas ini memiliki peran penting dalam menangkap data yang dikirim oleh server dan mempermudah akses ke daftar film dalam bentuk list, sehingga bisa diproses lebih lanjut, ditampilkan ke pengguna, atau disimpan ke database lokal.

File **MovieRepository.kt**

File **MovieRepositoryImpl.kt** berfungsi sebagai implementasi konkret dari antarmuka **MovieRepository**, yang bertanggung jawab mengelola pengambilan data

film populer dari sumber data jarak jauh (*remote*) maupun lokal (*local cache*). Kelas ini mengintegrasikan dua komponen utama, yaitu **TmdbApiService** sebagai API client untuk berkomunikasi dengan TMDB dan **MovieDao** untuk operasi basis data lokal melalui Room. Ia juga menerima **apiKey** sebagai kunci akses ke layanan TMDB.

Dalam implementasinya, fungsi **getPopularMovies()** mengembalikan sebuah aliran data (**Flow**) bertipe **Result<List<Movie>>**. Pertama, data dari database lokal diambil dan diubah menjadi domain model menggunakan fungsi ekstensi **toDomainMovie()**, lalu dikirim sebagai **Result.Success** jika tidak kosong. Ini memastikan pengguna tetap bisa melihat data meskipun belum ada jaringan.

Kemudian, repository mencoba memuat data terbaru dari API. Jika berhasil (**isSuccessful**), daftar film (**MovieDto**) dikonversi ke domain model, lalu di-cache ke database lokal setelah menghapus data sebelumnya. Hasil ini kembali dikirim sebagai **Result.Success**. Namun, bila terjadi kegagalan, seperti **HttpException**, **IOException**, atau error lainnya, error yang sesuai dibungkus dalam **Result.Error** dengan pesan deskriptif.

File Movie.kt

File **Movie.kt** dalam paket **domain.model** mendefinisikan struktur data **Movie** sebagai representasi entitas film dalam lapisan domain aplikasi. Kelas ini ditandai dengan anotasi **@Parcelize**, yang memungkinkan objek **Movie** untuk di-serialize secara otomatis ke dalam bentuk **Parcelable**. Hal ini sangat penting dalam konteks Android karena memungkinkan objek **Movie** dikirim antar komponen seperti **Activity** atau **Fragment** melalui **Intent** atau **Bundle** dengan efisien dan aman.

Struktur **Movie** terdiri dari beberapa properti inti, yaitu **id** sebagai identitas unik film, **title** sebagai judul film, **overview** sebagai ringkasan cerita, **posterPath** sebagai jalur gambar poster (yang bersifat opsional), **releaseDate** untuk tanggal rilis, serta **voteAverage** sebagai nilai rating rata-rata dari pengguna.

File GetPopularMoviesUseCase.kt

File **GetPopularMoviesUseCase.kt** merupakan bagian dari lapisan domain dalam arsitektur aplikasi dan berfungsi sebagai use case atau *interactor* yang menghubungkan logika bisnis dengan data yang diperlukan oleh UI. Kelas **GetPopularMoviesUseCase** memiliki tanggung jawab untuk menjalankan proses pengambilan data film populer dengan memanfaatkan fungsi yang disediakan oleh **MovieRepositoryImpl.kt**.

Dengan menggunakan operator **invoke()**, kelas ini memungkinkan *instance*-nya dipanggil seperti fungsi, sehingga memudahkan penggunaannya di kelas lain seperti **ViewModel**. Fungsi **invoke()** mengembalikan objek **Flow** dari **Result<List<Movie>>**, yang berarti aliran data film populer akan dipancarkan dalam bentuk hasil yang dapat berupa **Loading**, **Success**, atau **Error**. Hal ini selaras dengan pendekatan reaktif menggunakan Kotlin Coroutines dan **Flow**, sehingga proses pemanggilan data tetap responsif dan efisien terhadap perubahan data maupun kondisi jaringan.

File GetPopularMoviesUseCase.kt

File **GetPopularMoviesUseCase.kt** merupakan bagian dari lapisan domain dalam arsitektur aplikasi dan berfungsi sebagai use case atau *interactor* yang menghubungkan logika bisnis dengan data yang diperlukan oleh UI. Kelas **GetPopularMoviesUseCase** memiliki tanggung jawab untuk menjalankan proses pengambilan data film populer dengan memanfaatkan fungsi yang disediakan oleh **MovieRepositoryImpl.kt**.

Dengan menggunakan operator **invoke()**, kelas ini memungkinkan *instance*-nya dipanggil seperti fungsi, sehingga memudahkan penggunaannya di kelas lain seperti **ViewModel**. Fungsi **invoke()** mengembalikan objek **Flow** dari **Result<List<Movie>>**, yang berarti aliran data film populer akan dipancarkan dalam bentuk hasil yang dapat berupa **Loading**, **Success**, atau **Error**. Hal ini selaras dengan pendekatan reaktif menggunakan Kotlin Coroutines dan **Flow**, sehingga proses pemanggilan data tetap responsif dan efisien terhadap perubahan data maupun kondisi jaringan.

File `DetailActivity.kt`

File **`DetailActivity`** adalah bagian dari lapisan presentasi dalam aplikasi dan bertanggung jawab untuk menampilkan detail dari sebuah objek **`Movie`** yang dipilih oleh pengguna. Aktivitas ini menggunakan **`View Binding`** melalui **`ActivityDetailBinding`** untuk mengakses elemen UI secara langsung dan aman tanpa perlu memanggil **`findViewById`**. Saat **`onCreate()`** dijalankan, **`DetailActivity`** akan mencoba mengambil data **`Movie`** yang dikirim melalui **`Intent`** menggunakan kunci **`EXTRA_MOVIE`**. Untuk memastikan kompatibilitas dengan berbagai versi Android, pengecekan dilakukan menggunakan **`Build.VERSION.SDK_INT`**, di mana metode pemanggilan **`getParcelableExtra`** akan menyesuaikan berdasarkan versi SDK.

Jika data **`Movie`** berhasil ditemukan, maka informasi seperti judul, tanggal rilis, rating, dan sinopsis akan ditampilkan pada tampilan yang sesuai. Gambar poster film juga dimuat menggunakan library **`Glide`** dari path URL **`https://image.tmdb.org/t/p/w500`**, yang merupakan standar path poster dari API TMDB. Judul **`ActionBar`** akan diubah agar sesuai dengan judul film yang ditampilkan. Namun jika objek **`Movie`** bernilai **`null`**, pengguna akan diberi notifikasi berupa **`Toast`** yang menyatakan bahwa film tidak ditemukan, dan aktivitas langsung ditutup menggunakan **`finish()`**.

Selain itu, aktivitas ini juga menangani event saat tombol back di **`ActionBar`** ditekan dengan menimpa fungsi **`onOptionsItemSelected`**, yang memanggil **`onBackPressedDispatcher.onBackPressed()`** agar pengguna dapat kembali ke layar sebelumnya dengan cara yang sesuai dengan sistem.

File `MovieAdapter.kt`

File **`MovieAdapter`** merupakan implementasi dari **`ListAdapter`** yang dirancang untuk menampilkan daftar film (**`Movie`**) dalam komponen **`RecyclerView`**. Dengan menggunakan **`DiffUtil.ItemCallback`**, adapter ini secara efisien membandingkan perbedaan antar item dalam daftar, sehingga hanya elemen yang berubah saja yang akan diperbarui pada UI—meningkatkan performa terutama saat data diperbarui secara dinamis.

Adapter ini memiliki fungsi **onItemClick**, yaitu lambda opsional yang dapat diatur untuk menangani event ketika pengguna menekan tombol detail pada setiap item film. Pada metode **onCreateViewHolder**, layout item untuk tiap film diinflasi dari **ItemMovieBinding**, yang kemudian digunakan dalam **MovieViewHolder**. Dalam metode **onBindViewHolder**, data film pada posisi tertentu di-bind ke **ViewHolder** dengan memanggil fungsi **bind()**.

Kelas **MovieViewHolder** bertanggung jawab mengisi data ke elemen UI pada tiap item film, seperti judul, tanggal rilis, rating, dan sinopsis. Selain itu, poster film dimuat menggunakan library **Glide** dari URL **<https://image.tmdb.org/t/p/w500>**, yang merupakan path standar gambar dari API TMDB. Pada bagian **init**, tombol detail disiapkan untuk men-*trigger* fungsi **onItemClick** saat ditekan, sehingga event klik bisa ditangani di luar adapter, misalnya untuk navigasi ke halaman detail.

File MovieViewModel.kt

File **MovieViewModel** adalah bagian dari lapisan presentasi dalam arsitektur MVVM (Model-View-ViewModel) yang bertanggung jawab untuk mengelola data film populer dan menyediakannya ke antarmuka pengguna (**UI**). ViewModel ini menerima dependensi berupa **GetPopularMoviesUseCase**, yang berfungsi untuk menjalankan logika bisnis dalam mengambil data film dari repository.

Properti **_popularMovies** adalah objek **MutableLiveData** bertipe **Result<List<Movie>>**, yang menampung status data film populer, baik itu dalam bentuk **Loading**, **Success**, maupun **Error**. Data ini dibungkus menggunakan **LiveData** publik (**popularMovies**) agar dapat diamati oleh UI tanpa memberi akses untuk memodifikasi nilainya secara langsung, menjaga prinsip encapsulation.

Saat **ViewModel** diinisialisasi, fungsi **fetchPopularMovies()** akan langsung dijalankan. Fungsi ini memanfaatkan coroutine pada **viewModelScope** untuk mengeksekusi use case secara asynchronous, lalu mengumpulkan (**collect**) hasilnya. Setiap perubahan hasil (loading, data berhasil, atau error) akan diperbarui ke LiveData **_popularMovies**, yang nantinya akan diterima oleh observer di UI seperti **Activity** atau **Fragment**.

File ViewModelFactory.kt

File **ViewModelFactory** pada kode tersebut berfungsi sebagai pabrik pembuat objek **ViewModel**, khususnya **MovieViewModel**, yang membutuhkan dependensi eksternal yaitu **GetPopularMoviesUseCase**. Karena **ViewModel** tidak secara langsung mendukung konstruktor berparameter, maka **ViewModelProvider.Factory** digunakan untuk menyuntikkan dependensi secara manual saat **ViewModel** dibuat.

Metode **create()** dalam **ViewModelFactory** akan dipanggil oleh **ViewModelProvider** saat sistem membutuhkan instance **MovieViewModel**. Ia memeriksa apakah **modelClass** merupakan turunan dari **MovieViewModel** menggunakan **isAssignableFrom**. Jika cocok, maka instance **MovieViewModel** dikembalikan dengan cara casting ke tipe generik **T**. Jika tidak, akan dilempar **IllegalArgumentException** karena **ViewModelFactory** ini hanya bisa membuat **MovieViewModel**.

File Result.kt

File **Result** dalam kode tersebut merupakan sebuah *sealed class* yang dirancang untuk merepresentasikan status dari suatu operasi asynchronous, seperti pengambilan data dari API atau database, dalam arsitektur aplikasi berbasis MVVM. Kelas ini memiliki tiga kemungkinan status, yaitu **Loading**, **Success**, dan **Error**. Status **Loading** digunakan untuk menunjukkan bahwa proses sedang berlangsung, biasanya dimanfaatkan untuk menampilkan indikator pemuatan di antarmuka pengguna. Status **Success** membawa data hasil dari operasi yang berhasil, di mana data tersebut akan diteruskan ke UI untuk ditampilkan, misalnya daftar film populer dalam bentuk **List<Movie>**. Sedangkan status **Error** digunakan untuk menangani kondisi ketika operasi gagal, misalnya karena kesalahan jaringan, kesalahan parsing, atau kondisi tak terduga lainnya, dengan membawa objek **Exception** yang memuat pesan error.

File activity_detail.xml

Kode **Activity_Detail.xml** merupakan layout untuk **DetailActivity** pada aplikasi Android yang menampilkan detail sebuah film. Layout ini menggunakan **ScrollView** sebagai elemen induk agar kontennya dapat digulir jika melebihi tinggi layar perangkat. Di dalam **ScrollView**, terdapat **ConstraintLayout** yang mengatur posisi elemen-elemen UI secara fleksibel berdasarkan batasan (constraints).

Elemen pertama adalah **ImageView** dengan ID **iv_detail_poster** yang digunakan untuk menampilkan poster film dalam ukuran besar dan memenuhi lebar layar. Setelah itu, ada beberapa **TextView** yang menyajikan informasi detail film, seperti judul (**tv_detail_title**), tanggal rilis (**tv_detail_release_date**), dan rating (**tv_detail_vote_average**). Di bagian bawah, terdapat label "Overview" (**tv_detail_overview_label**) dan **TextView** (**tv_detail_overview**) yang berfungsi untuk menampilkan deskripsi lengkap atau sinopsis film. Setiap elemen diberi margin dan diatur dengan constraint agar rapi dan responsif terhadap berbagai ukuran layar.

File activity_main.xml

Kode **activity_main.xml** merupakan **MainActivity** yang menampilkan daftar film populer. Layout ini menggunakan **ConstraintLayout** sebagai kontainer utama agar setiap komponen UI dapat diatur secara fleksibel dan responsif. Di bagian atas terdapat **TextView** dengan ID **tv_title** yang berfungsi sebagai judul halaman dengan teks "Popular Movies" dan tampilan tebal berukuran besar. Tepat di bawahnya, terdapat elemen **ProgressBar** dengan ID **progress_bar** yang secara default disembunyikan (**visibility="gone"**), dan digunakan untuk menandakan bahwa data sedang dimuat.

Jika terjadi kesalahan saat mengambil data, maka **TextView** dengan ID **tv_error** akan ditampilkan bersama **Button** dengan ID **btn_retry**, yang memungkinkan pengguna untuk mencoba memuat ulang data. Komponen utama dari halaman ini adalah **RecyclerView** dengan ID **rv_movies** yang akan menampilkan daftar film dalam bentuk list, dan posisinya berada di bawah judul dan mengisi sebagian besar area layar. Di pojok kanan atas, terdapat **SwitchMaterial** dari *Material Components* dengan ID

switch_dark_mode yang memungkinkan pengguna untuk mengaktifkan atau menonaktifkan mode gelap (*dark mode*).

File **item_movie.xml**

Kode **item_movie.xml** merupakan layout item untuk RecyclerView yang digunakan pada daftar film populer. Komponen utamanya adalah **CardView**, yang memberikan tampilan berbentuk kartu dengan sudut melengkung (**cardCornerRadius="8dp"**) dan efek bayangan (**cardElevation="4dp"**) agar terlihat menonjol dari latar belakang. Di dalam **CardView** terdapat **ConstraintLayout** yang mengatur posisi setiap elemen secara fleksibel.

Setiap item terdiri dari sebuah **ImageView** dengan ID **iv_poster** di sisi kiri yang menampilkan gambar poster film. Di sebelah kanannya terdapat tiga **TextView** berturut-turut: **tv_movie_title** untuk menampilkan judul film dengan ukuran dan gaya tulisan yang menonjol, **tv_release_date** untuk menampilkan tanggal rilis, dan **tv_vote_average** yang menampilkan rating film. Di bawah poster, terdapat **TextView** lain dengan ID **tv_overview** yang menampilkan sinopsis singkat film, dibatasi maksimal 3 baris dan dipotong (**ellipsize="end"**) jika terlalu panjang agar tetap ringkas. Terakhir, ada sebuah tombol **Button** dengan teks "**Lihat Detail**" yang memungkinkan pengguna untuk melihat informasi lebih lengkap dari film yang bersangkutan.

File **build.gradle.kts (Module:app)**

Kode **build.gradle.kts** merupakan konfigurasi utama untuk aplikasi Android **MovieList** yang ditulis menggunakan Kotlin DSL. Plugin yang digunakan mencakup plugin Android aplikasi, Kotlin Android, **kotlin-parcelize** untuk mendukung parcelable class, **kotlin-kapt** untuk annotation processing (misalnya untuk Room), serta plugin Kotlin serialization untuk pengolahan data JSON secara efisien menggunakan Kotlinx Serialization.

Bagian **android** mengatur informasi penting seperti namespace, versi SDK (**compileSdk**, **minSdk**, dan **targetSdk**), serta konfigurasi build default seperti

applicationId, **versionCode**, dan **versionName**. Selain itu, juga disetel opsi build khusus untuk rilis seperti penggunaan ProGuard, konfigurasi **Java 11**, dan aktivasi fitur **viewBinding** untuk mempermudah akses ke komponen UI dari XML.

Di bagian **dependencies**, tercantum berbagai pustaka penting. Untuk antarmuka dasar, digunakan AndroidX Core, AppCompatActivity, Material Design, Activity, dan ConstraintLayout. Pengujian didukung oleh JUnit, Espresso, dan library pengujian AndroidX. Untuk penyimpanan lokal, digunakan Room (termasuk **room-runtime**, **room-ktx**, dan compiler dengan **kapt**). Untuk pemuatan gambar, digunakan Glide. Pengambilan data dari API dilakukan melalui Retrofit, dikombinasikan dengan Kotlinx Serialization dan OkHttp sebagai klien HTTP. Library **lifecycle** dipakai untuk arsitektur berbasis ViewModel dan LiveData. **Coroutine** juga digunakan untuk pengelolaan thread asynchronous yang efisien. Terakhir, terdapat dukungan navigasi menggunakan **androidx.navigation** untuk berpindah antar-fragment secara aman.

D. Tautan Git

<https://github.com/biilaaa/Pemrograman-Mobile>